

Trivial relocatability options

Proposal for an alternative approach to trivial relocatability

Document #: P2786R0
Date: 2023-02-10
Project: Programming Language C++
Audience: Evolution Incubator
Library Evolution Incubator
Revises: N/A
Reply-to: Mungo Gill
<mgill83@bloomberg.net>
Alisdair Meredith
<ameredith1@bloomberg.net>

Contents

1	Abstract	2
2	Revision history.	3
2.1	R0: Issaquah 2023	3
3	Introduction	3
4	Motivating use cases	3
4.1	Efficient <code>vector</code> growth	3
4.2	Moving types without empty states	3
4.3	<code>pmr</code> types are often trivially relocatable	4
4.4	Future proposal for language support for allocators	4
5	Experience at Bloomberg	4
6	Key differences between the proposals	4
6.1	Trivial implies semantics, not syntax	4
6.2	Predictable specification without deference to QoI	4
6.3	Can <code>std::swap</code> be implemented as three relocates?	5
6.4	Support for <code>pmr</code> types	5
7	Proposed changes	5
7.1	New terms and definitions	5
7.2	New type category	6
7.3	New semantics	6
7.4	New type trait	7
7.5	New syntax	7
7.6	Diagnosable errors	8
7.7	Relocation functions	11
8	Design choices	13
8.1	No library support is mandated	13
8.2	Contextual keyword vs. attribute	13
8.3	Type trait vs. concept	14
8.4	<code>trivially_relocate</code> as the single place for compiler magic	14

9	Known concerns	15
9.1	Separately managed objects	15
9.2	Internal pointers to members	15
9.3	Active element of a union	15
9.4	ABI compatibility	16
9.5	Relocating <code>const</code> -objects	16
9.6	Trivially relocatable is not trivially swappable	16
10	Alternative designs	17
10.1	Smarter default for dependent templates	17
10.2	Ignoring <code>trivially_relocatable</code> like <code>constexpr</code>	17
11	FAQ	17
11.1	Is <code>void</code> trivially relocatable?	17
11.2	Are reference types trivially relocatable?	18
11.3	Why not?!	18
11.4	Why can a class with a reference member be trivially relocatable?	18
11.5	Are <i>cv</i> -qualified types, notably <code>const</code> types, trivially relocatable?	18
11.6	Can <code>const</code> -qualified types be passed to <code>trivially_relocate</code> ?	18
11.7	Can non-implicit-lifetime types be trivially relocatable?	18
11.8	Why are virtual base classes not trivially relocatable?	18
11.9	Why do deleted special members inhibit implicit trivial relocatability?	18
12	Proposed wording	19
12.1	Feature macros	19
12.2	Specification of trivial relocatability	19
12.3	Grammar for <code>trivially_relocatable</code>	20
12.4	New type trait	20
12.5	Relocation functions	21
13	Appendix: Comparison with [P1144R6]	23
13.1	A Common Basis	23
13.2	Difference in tone	23
13.3	Interface vs. semantics	24
13.4	Trivial copyability implies trivial relocatability	24
13.5	Implicit trivial relocatability	24
13.6	Detecting miscategorization	25
13.7	Moved object lifetime and destruction	25
13.8	Support for <code>const</code> data members	26
13.9	New relocation functions	27
13.10	Cosmetic differences	29
14	Acknowledgements	30
15	References	30

1 Abstract

This paper examines an approach to support trivial relocatability, building upon ideas in previous papers [P1029R3] and [P1144R6], and leveraging the experience of supporting *bitwise movability* in the BDE library. It embraces the motivation for such a feature in those papers, while providing what we believe to be a more rigorous design and specification.

2 Revision history.

2.1 R0: Issaquah 2023

Initial draft of the paper.

3 Introduction

For our purposes, a *trivial relocation operation* is a *bitwise copy* that ends the lifetime of its source object, as-if its storage were used by another object (6.7.3 [basic.life]p5). Importantly, nothing else is done to the source object, in particular **its destructor is not run**. This operation will typically (though exceptions are not forbidden) be semantically equivalent to a move construction immediately followed by a destruction of the source object.

Any trivially copyable type is *trivially relocatable* by default. Many other types, even those which have non-trivial move constructors and destructors, can maintain their correct behavior when trivially relocated — skipping the source object’s destructor allows for skipping all bookkeeping updates that might need to be done by the target object’s move constructor. This includes many resource-owning types, such as `vector`, `unique_ptr`, and `shared_ptr`.

Note that simply doing a bitwise copy of these non-trivially-copyable objects will, as of C++23, result in undefined behavior (when the copied bytes are treated by later code as an object of the original type). Making this operation well-defined for those types which opt into this behavior is the primary goal of proposing this feature as a language extension. The secondary goal is to implicitly support a wider range of trivially relocatable types. The tertiary goal is to provide better diagnostics when trivial relocation semantics are misused.

We will also perform a detailed comparison of the differences with [P1144R6] (see appendix). Given the level of similarity between the two papers, key differences will be noted where relevant.

4 Motivating use cases

We believe [P1144R6] has done a good job a motivating proposals in this area. Here, we highlight the specific use cases that drive our proposal, just as a reinforcement of the earlier paper.

4.1 Efficient vector growth

Suppose we have a move-only type, `class MoveOnlyType` (for example, a unique ownership smart pointer), and we wish to hold a vector of these types `std::vector<MoveOnlyType>`. Simply emplacing 5 of these objects would require that `MoveOnlyType`’s move constructor and destructor be called 7 additional times due to the vector expansion required as more elements are inserted than the capacity (at least in one current implementation of `std::vector`).

If `MoveOnlyType` were trivially relocatable, and if `std::vector` were to take that into account as an optimization, then the vector expansion caused by these 5 emplacements would require only 3 `memmove` operations, with no additional calls to `MoveOnlyType`’s move constructor and destructor.

For this example we are assuming an initially empty `vector` with no reserve capacity, and that the implementation has a growth strategy of doubling the reserved space when more is required, from 0 to 1 to 2 to 4 to 8.

4.2 Moving types without empty states

Some types do not have a non-allocating empty state, so cannot have a `noexcept` move constructor. One example is a known implementation strategy for `std::list` that always allocates at least a sentinel node. Lacking a non-throwing move constructor, vectors of such list have a painful growth strategy. However, as long as the sentinel does not maintain a back-pointer into its list object, such a type can be trivially relocated as the old object

immediately ends its life without running its destructor, so does not have to restore invariants — there is no window of opportunity to access the live object in a state where it has broken invariants.

4.3 `pmr` types are often trivially relocatable

The original motivation for this feature in the BDE library was to ensure efficient movement of allocator aware types, using the allocator model that became standardized in namespace `std::pmr`. As the allocator is simply a pointer to a memory resource, and allocated memory does not reside within the owning object itself, many non-trivial allocator-aware types can be trivially relocatable if an appropriate markup is available.

4.4 Future proposal for language support for allocators

The authors are also working on a separate proposal for direct language support for allocators, based upon the `std::pmr` design ([P2685R0]). That proposal anticipates support for trivial relocatability.

5 Experience at Bloomberg

Bloomberg has relied heavily on low level optimizations enabled by assuming the trivially relocatable model holds. This implementation experience is built on the, so far valid, assumption that no current compilers are optimizing to transform programs based on the specific undefined behaviors we exploit. The emulation is achieved through a type trait, `bslmf::IsBitwiseMovable`. More recently, in an experimental branch to explore language extensions, `pbstd::is_trivially_relocatable` is used to demonstrate relocation of types using `std::pmr::polymorphic_allocator`. This experimental model is a pure library extension, and has no impact unless libraries are written to test this trait before choosing an optimized implementation. In particular, types that are not trivially copyable must opt into the trait with a special traits markup, or by specializing the trait for their relocatable type. Note that user specialization would not be permitted for a standardized type trait, per 21.3.2 [meta.rqmts]p4.

The initial language support we propose is that the new trait will detect trivially copyable types as also being trivially relocatable by default, while other types will default to non-trivially-relocatable. This part can be covered by a library emulation, implementing the new trait in terms of `std::is_trivially_copyable`.

6 Key differences between the proposals

In reading the material that follows, it is good to be aware of the major differences in its design that led to details differing from [P1144R6].

6.1 Trivial implies semantics, not syntax

[P1144R6] is based upon relocatability semantics, which are equivalent to a move followed by destruction. That design leads to the deduced relocatable property being framed in terms of publicly accessible move constructor and destructor, and the trivially relocatable requirements are based on those same syntactic requirements.

This proposal leads with the idea that triviality is key, and that all other trivial semantics in the language are based upon the trivial semantics of bases and members, not syntax. Hence, public access to relocating functions does not matter, as we will relocate using the trivial semantic instead for such types. For example, this leads to support for trivially relocating friends.

6.2 Predictable specification without deference to QoI

[P1144R6] provides permission to use bitwise copies to perform relocation operations, but does not mandate it. That optimization is left as a QoI feature of the relocating library functions. Misuse by annotating types that are not suitable for relocation often leads to UB, or programs that are ill-formed, no diagnostic required.

This proposal leaves no room for QoI, fully specifying observable behavior, and requiring (diagnosable) ill-formed programs when the facility is misused. This principle of predictability in the specification/syntax leads to a divergence over whether the proposed property can be explicitly revoked.

6.3 Can `std::swap` be implemented as three relocates?

[P1144R6] constrains itself to supporting only types that support implementing `std::swap` as three relocate operations. That rules out support for types that are important to this proposal, notably all `pmr` containers such as `std::pmr::vector<std::pmr::string>`. See [Trivially relocatable is not trivially swappable](#) for details.

6.4 Support for `pmr` types

The scoped allocator model, typified by `std::pmr` types, is a strong motivator behind this proposal, notably for examples like `std::pmr::vector<std::pmr::string>`. As such, we are careful to specify the feature to support our primary use case.

We do not want to state the motivations of [P1144R6] as we are not the authors, so any summary of ours would misstate in some way. However, it appears to be more concerned with supporting types with some notion of regular behavior, and as such does not support types following the `pmr` allocator model.

7 Proposed changes

Our proposal changes and extends C++23 as follows.

7.1 New terms and definitions

We need to introduce and specify some key terms. These terms can be found in numerous other proposals, and the definitions proposed here are very similar:

- **relocate**: To **relocate** a type from memory address `src` to memory address `dst` means to perform an operation or series of operations such that an object equivalent (often identical) to that which existed at address `src` exists at address `dst`, that the lifetime of the object at address `dst` has begun, and that the lifetime of the object at address `src` has ended.
- **relocatable**: To say that an object is **relocatable** is to say that it is possible to **relocate** the object from one location to another.
- **trivially relocatable**: Conceptually, a type is **trivially relocatable** if it can be **relocated** by means of copying the bytes of the object representation and then ending the lifetime of the original object without running its destructor.

A **trivially relocatable** type is a type that is **implicitly trivially relocatable** and/or is **explicitly trivially relocatable**, and/or is an array of **trivially relocatable** types; otherwise the type is not **trivially relocatable**. Any otherwise **trivially_relocatable** type can be declared non-**trivially relocatable** by means of the `trivially_relocatable` keyword with value `false`.

- **implicitly trivially relocatable**: An **implicitly trivially relocatable** type has no user-provided or deleted destructors, no virtual base classes, no non-trivially-relocatable bases or non-static data members, and the constructor selected for construction from a single rvalue of the same type is neither user-provided nor deleted.
 - Examples of types that are trivially relocatable by default are trivially copyable types, such as scalar types, aggregates of trivially relocatable types, including arrays of such types, and such aggregates with `const` and/or reference data members. Empty types can satisfy the requirements for an **implicitly trivially relocatable** type.
 - Assuming it meets the requirements of the previous bullet point, then if a class has an appropriate (move or copy) constructor, it is **implicitly trivially relocatable** regardless of the access level

(`public/protected/private`) of that constructor. This non-requirement for accessibility follows the same model as the standard specification for **trivially copyable** types.

- If the copy constructor inhibits the declaration of the move constructor, then a class is implicitly trivially relocatable if the copy constructor is implicitly defined, otherwise the copy constructor is not relevant. Note that there are no requirements that the destructor be accessible, merely that it is neither deleted nor user-provided.
- **explicitly trivially relocatable**: An **explicitly trivially relocatable** type is a user defined (class) type that is defined with the contextual keyword `trivially_relocatable` and value `true`, with the following proviso:
 - An **explicitly trivially relocatable** class type may not contain any non-**trivially relocatable** non-static data members nor base classes, nor have any virtual base classes. (I.e., it is a diagnosable error to add the keyword with value `true` to a class that does not qualify.)

It is important to note that we are proposing to permit, by means of said keyword, types that would otherwise be non-copyable and non-movable to be **relocatable**. For this reason we cannot define **relocate** and **relocatable** in terms of a move construction followed by a destruction (as is done in [P1144R6]). The ability to explicitly make a type trivially relocatable enables providing a customized (and thus non-trivial) move constructor and destructor while declaring that the compound operation is trivial.

7.2 New type category

To better integrate language support, we further recommend that the language can detect types as **trivially relocatable** where all their bases and non-static data members are, in turn, **trivially relocatable**; the constructor selected for construction from a single rvalue of the same type is neither user-provided nor deleted; and their destructor is neither user provided nor deleted. This definition follows the same principle used in the standard to define trivially copyable.

7.3 New semantics

In order to ensure that libraries taking advantage of the trivially relocatable semantic do not introduce **undefined behaviour**, the model of lifetimes for objects must be extended to allow for relocation of **trivially relocatable** types. As the compiler cannot know if a specific `memcpy` or `memmove` call is intended to duplicate or move an object, we propose introducing the `trivially_relocate` function template to call `memmove` on our behalf, which signifies to the compiler and other source analysis tools that the lifetime of the new object(s) has begun and the lifetime of the original object(s) has ended:

```
template <class T>
requires is_trivially_relocatable_v<T>
T* trivially_relocate(T* begin, T* end, T* new_location) noexcept;
```

Note that this function is designed to move a range of objects, rather than a single object, as that is expected to be the common use case. Further note that, consistent with its low level purpose often tied to move semantics, this function is denoted with `noexcept` despite having a narrow contract regarding valid and reachable pointers.

This design deliberately puts all “compiler magic” and core-language interaction dealing with the object lifetimes into a single place, rather than into a number of different **relocate**-related overloads. Note that there is no permission for a user to copy the bytes to perform a relocation themselves, unlike with trivial copyability, although that would still work for trivially copyable types.

`trivially_relocate` can be thought of as ending the lifetime of the moved-from objects, followed by a `memmove`, followed by `start_lifetime_as` (or maybe `start_lifetime_as_array`) on the moved-to objects. Unlike `memmove` on its own, it is restricted to trivially relocatable types rather than to implicit lifetime types.

Note that `start_lifetime_as` is constrained to work only for implicit lifetime types whereas this proposal is intended to support all trivially relocatable types, which are often not implicit lifetime types. The different constraints are appropriate in each case. For the currently specified `start_lifetime_as` function, the idea is

that we point the compiler to a region of memory, and say “take these bytes of unknown provenance and turn them into objects”. In particular, we might be copying bytes into memory from a stream, and those bytes did not originate as objects in *this* abstract machine. Conversely, `trivially_relocate` takes existing valid objects in memory, copies their bytes to a new location, and asks the compiler to imbue life into specifically those bytes copied from known valid objects. It is important that the copying and imbuing life occur within the same transaction, as that gives the compiler its necessary guarantees. Hence, all the new functionality is bundled into a single `trivially_relocate` function, rather than decomposing into smaller parts that would allow the users to perform the `memmove` themselves.

7.4 New type trait

In order to expose the relocatability property of a type to library functions seeking to provide appropriate optimizations, we propose a new trait `std::is_trivially_relocatable<T>` which enables the detection of **trivial relocatability**.

```
template< class T >
struct is_trivially_relocatable;

template< class T >
inline constexpr bool is_trivially_relocatable_v = is_trivially_relocatable<T>::value;
```

having a base characteristic of `std::true_type` if `T` is **trivially relocatable** and `std::false_type` otherwise.

Note that it is expected that the `std::is_trivially_relocatable` trait shall be implemented through a compiler intrinsic, much like `std::is_trivially_copyable`, so the compiler can use that intrinsic when the language semantics require trivial relocatability, rather than requiring actual instantiation (and knowledge) of the standard library trait.

7.5 New syntax

In order to enable trivial relocatability to be useful for more complicated (i.e., non-**trivially copyable**) types, it must be possible to explicitly mark non-**trivially copyable** types as **trivially relocatable**. As this should be an issue only for class types (including unions), for which we recommend adding a new contextual keyword `trivially_relocatable` as part of the class definition, similar to how `final` applies to classes. E.g.,

We propose one new contextual keyword that can be placed in a class-head to attach a trivially relocatable predicate to a class:

- `trivially_relocatable(bool-expression)` which is used:
 - With value `true` to explicitly make a class **trivially relocatable**, and
 - With value `false` to explicitly remove **trivial relocatability** from a class.

The boolean predicate is optional, with a plain `trivially_relocatable` defaulting to `true`.

It is possible, by means of the `trivially_relocatable(bool-expression)` specification, to declare a class as **trivially relocatable** even if that class has a user-defined copy constructor and/or move constructor and/or destructor. This differs from [P1144R6] in the following notable ways:

- Where `trivially_relocatable` is specified with value `true`, we do not require that the move constructor, copy constructor, and/or destructor be public or unambiguous. The `trivially_relocatable` specification takes precedence.
- It is possible to render, by means of the keyword and value `false`, any type, even a **trivially copyable** type, non-**trivially relocatable**.

Our motivation for the explicit specification *always* supplanting the implicit specification, rather than just the case of `true` supplanting `false`, is the confusion we encountered when considering other semantics in [alternative designs](#) below. It became clear that it was much simpler to reason about our examples when the trivial relocation specification could be trusted to mean literally what it said.

While we do not have use cases for making trivially relocatable types to be non trivially relocatable, we do not have use cases to disallow it either, and must choose a meaning for the syntax. Our experience with language design in general has been that users will find corners where even the most obscure feature is useful, so prefer to not remove a potentially useful feature that is intuitive from the syntax.

It may be argued that this is a case where [P1144R6] leans on the semantics of the feature where this proposal leans on the syntax.

7.6 Diagnosable errors

In a non-dependant context, it would be a diagnosable error to mark a type as **trivially relocatable** if it comprises any bases or non-static members that are not **trivially relocatable**. Types with virtual base classes are automatically not **trivially relocatable**, as their implementation on some platforms involves an internal pointer. We prefer that this low level behavior is consistent across platforms, rather than left as an unspecified QoI concern, as our current experience has not yet turned up a usage of virtual base classes that would also benefit from this feature.

Note that there are no issues with virtual functions, as virtual function table implementations do not take a pointer back into the class, so the vtable pointer can be safely relocated.

7.6.1 Simple examples without a predicate

The common form is expected to be the simple case, without a predicate.

```
struct MyType trivially_relocatable : BaseType {
    // class definition details

    MyType(MyType&&); // user supplied
    // Having a user-provided move constructor, `MyType` would not be
    // trivially relocatable by default. The `trivially_relocatable`
    // annotation trusts the user that this type can indeed be trivially
    // relocated.
};

struct NotRelocatable : BaseType {
    // class definition details

    NotRelocatable(NotRelocatable&&); // user supplied
    // Having a user-provided move constructor, `NotRelocatable` is not
    // trivially relocatable.
};

struct Error trivially_relocatable : BaseType {
    NotRelocatable member;
    // This class is ill-formed, as it requests to be trivially relocatable,
    // but the compiler can see a non-relocatable data member that cannot be
    // worked around.

    Error(Error&&); // user supplied
    // There is nothing this move constructor can do to repair the trivial
    // relocatability property, as it is not invoked during trivial
    // relocation.
};
```


7.6.2 Examples using the predicate

The boolean predicate form, `trivially_relocatable(false)`, can be used to opt out of the behavior for a type that might otherwise be **trivially relocatable** by default. However, the main purpose of the predicate is to allow class templates to indicate their trivial relocatability where their opt-in might depend on the supplied template arguments.

For example purposes, let us consider the following two classes:

```
struct Relocatable trivially_relocatable(true) {}; // trivially relocatable
struct Alternative trivially_relocatable(false) {}; // not trivially relocatable

static_assert( is_trivially_relocatable_v<Relocatable> );
static_assert(!is_trivially_relocatable_v<Alternative> );
```

Clearly, `Relocatable` is a trivially relocatable class type, and `Alternative` is a non-trivially relocatable class type. We will use these classes to illustrate how similar, but subtly different, class templates behave.

As an initial example, we write a simple aggregate that demonstrates we get the expected behavior that correctly deduces trivial relocatability when we have no user-supplied special members:

```
template<class TYPE>
struct Example {
    TYPE value_a;
    TYPE value_b;
};

static_assert( is_trivially_relocatable_v<Example<Relocatable>> );
static_assert(!is_trivially_relocatable_v<Example<Alternative>> );
```

However, for most of our remaining examples we are concerned with the case of a class template that provides its own special members, so needs to supply a trivial relocation specification. The examples look simple, and may lead to thinking “why am I messing with all this template syntax when the simple `Example` works?” but remember, these are deliberately simplified examples to highlight just the relevant code, and the underlying lesson is intended for larger code in practice, where `Example` would clearly not suffice.

As our first example, we write a class template that uses the trivially relocatable specification to forward the trivial relocatability of its dependent members:

```
template<class TYPE>
class Duo trivially_relocatable(is_trivially_relocatable_v<TYPE>)
{
private:
    TYPE value_a;
    TYPE value_b;
public:
    ~Duo() {} // User provided destructor so not implicitly relocatable
};

static_assert( is_trivially_relocatable_v<Duo<Relocatable>> );
static_assert(!is_trivially_relocatable_v<Duo<Alternative>> );
```

Next, we use type constraints in a `requires` clause instead, so see how the behavior differs:

```
template<class TYPE>
    requires is_trivially_relocatable_v<TYPE>
class RelocatableDuo trivially_relocatable
{
private:
```

```

    TYPE value_a;
    TYPE value_b;
public:
    ~RelocatableDuo() {} // User provided destructor so not implicitly relocatable
};

static_assert( is_trivially_relocatable_v<RelocatableDuo<Relocatable>>>);
static_assert(!is_trivially_relocatable_v<RelocatableDuo<Alternative>>>); // ill-formed

```

Observe that the static assertion for `RelocatableDuo<Alternative>` is ill-formed not because that `static_assert` fails, but rather, that the `RelocatableDuo` template cannot be instantiated for `Alternative` at all, i.e., `RelocatableDuo` is a template that wraps only trivially relocatable types, and so can guarantee to always be trivially relocatable.

For another example, we can try to make a class template unconditionally trivially relocatable:

```

template<class TYPE>
class TryRelocatable trivially_relocatable
{
private:
    TYPE value_a;
    TYPE value_b;
public:
    ~TryRelocatable() {} // User provided destructor so not implicitly relocatable
};

static_assert( is_trivially_relocatable_v<TryRelocatable<Relocatable>>>);
static_assert(!is_trivially_relocatable_v<TryRelocatable<Alternative>>>); // ill-formed

```

The `Alternative` instantiation fails again, but this time it fails because the `trivially_relocatable` specification is violated, which is a diagnosable error. The error message is likely to refer to the `value_a` and `value_b` members, where the error message for the `RelocatableDuo` example would be related to violating the type constraints of the `requires` clause.

Note that as an unadorned `trivially_relocatable` specification is equivalent to `trivially_relocatable(true)`, we can also consider the opposite case, `trivially_relocatable(false)`:

```

template<class TYPE>
class NotRelocatable trivially_relocatable(false)
{
private:
    TYPE value_a;
    TYPE value_b;
public:
    ~NotRelocatable() {} // User provided destructor so not implicitly relocatable
};

static_assert(!is_trivially_relocatable_v<NotRelocatable<Relocatable>>>);
static_assert(!is_trivially_relocatable_v<NotRelocatable<Alternative>>>);

```

Here we see both instantiations are again valid, and the trivial relocation specification forces both instantiations to be not trivially relocatable.

As a final example of Duo-like types, we consider what happens if one of the members is not type-dependent, and not relocatable:

```
template<class TYPE>
struct Erroneous trivially_relocatable
{
    Alternative value_a; // ill-formed
    TYPE        value_b;
};
```

This case is ill-formed in all cases, and can be diagnosed in the template definition without waiting for an instantiation.

Another example where the trivial relocation specification might be useful is for trivial relocatability to be contingent on avoiding some small object optimization, such as:

```
template<class T>
class Container trivially_relocatable(sizeof(T) > SHORT_OPTIMIZATION_LIMIT)
{
    // Store small objects with an in-object representation, and dynamically
    // allocate storage for larger objects.

    // ...
};
```

Here we are concerned purely with whether a type is small enough to fit the small object optimization, and make no effect to further constrain on type. This might be how we approach retrofitting trivial relocatability into an existing library without raising ABI concerns.

7.7 Relocation functions

We propose two additional library functions to relocate ranges of objects. Note that this initial proposal does not have single-object relocation functions as our primary motivation is relocating objects in bulk. It would be easy to add single-object relocate functions, but the effect can be achieved by calling the proposed functions with a range of a single object, so we wait to hear that the evolution groups feel sufficiently motivated to request such convenience functions.

7.7.1 trivially_relocate

We propose the following function template to relocate trivially relocatable objects by means of a `memmove`. This function is the unique entry point into the core magic that tracks and manages object lifetimes in the abstract machine:

```
template <class T>
    requires (is_trivially_relocatable_v<T> && !is_const_v<T>)
void trivially_relocate(T* begin, T* end, T* new_location) noexcept;
```

This function template is equivalent to:

```
memmove(new_location, begin, sizeof(T) * (end - begin));
```

with the precondition that `end` is reachable from `begin`. It further has the following two very important effects, that matter to the abstract machine but do not have any apparent physical effect (i.e., these effects do not change bits in memory), much like `std::launder`:

- it begins the lifetime of the objects `*new_location`, `*(new_location+1)`, ..., through to `*(new_location+end-begin-1)`. If any of the objects or their subobjects are unions, they have the same active elements as the corresponding objects in the range `[begin, end)`.
- it ends the lifetime of the objects `*begin`, `*(begin+1)`, ..., through to `*(end-1)`. This means it will be *Undefined Behavior* to access these objects or to attempt to destruct any of them.

Note: the first bullet (beginning the new lifetime(s) of the new object(s)) could be achieved by saying that this is equivalent to:

```
memmove(new_location, begin, sizeof(T) * (end - begin));
std::start_lifetime_as_array_without_preconditions(new_location, sizeof(T) * (end - begin))
```

but there is not currently a mechanism to end the lifetime(s) of the source object(s).

7.7.2 relocate

We also propose a new “convenience” function template:

```
template <class T>
    requires ((is_trivially_relocatable_v<T> && !is_const_v<T>) ||
              is_nothrow_move_constructible_v<T>)
T* relocate(T* begin, T* end, T* new_location)
```

Which is equivalent to:

```
if constexpr (is_trivially_relocatable_v<T>) {
    trivially_relocate(begin, end, new_location);
}
else if (ranges-do-not-overlap) {
    std::uninitialized_move(begin, end, new_location);
    std::destroy(begin, end);
}
else {
    // relocate-and-destroy each member in the appropriate order
}
```

Note that this function supports overlapping ranges, just like `memmove`.

This function is similar to `uninitialized_relocate` in [P1144R6], except that our proposal requires pointers rather than input iterators for the source, and mandates we always trivially relocate types that support trivial relocation. The always-trivially-relocate-where-possible requires the input range be contiguous, but in principle we could relax this to using iterators that model the `contiguous_iterator` concept.

This function is also constrained to require nothrow move constructible types, as that better reflects its use case as an efficient relocation with minimal overhead. If an exception were thrown, the user would lack the information to put the program back into a good state, and the following `move_and_destroy` function is intended to support such use cases.

We do not have `uninitialized` in the name, as relocation already implies that we target range will be overwritten — but note that we *do* support overlapping ranges where some of the relocating objects are already initialized (and being overwritten) in the target range, which would therefore *not* be fully uninitialized.

7.7.3 move_and_destroy

We further propose a second “convenience” function template, that takes iterator ranges, supports potentially-throwing move constructors, but does *not* support overlapping input and output ranges:

```
template <class InputIterator, class NoThrowForwardIterator>
    requires is_nothrow_move_constructible_v<iter_value_t<NoThrowForwardIterator>>
NoThrowForwardIterator move_and_destroy(InputIterator first, InputIterator last,
                                       NoThrowForwardIterator destination);
```

This function is directly inspired by `uninitialized_relocate` in [P1144R6]. However, as per its name in this proposal, it is mandated to *always* perform a move-construct followed by destruction and is not given permission to switch to a trivially relocating implementation for certain types. Implementations may still find ways to

as-if such an implementation if they stare carefully at all the requirements, but we do not explicitly ban such implementations, we do not anticipate that as a worthwhile optimization.

Note that this function does not accept types that are not move constructible, even if they are trivially relocatable.

We do not support overlapping ranges in this function as, in general, it is undefined behavior to compare iterators into different sequences when trying to determine if there is an overlap, never mind the cost for non-random access iterators, and unsupportability of input iterators. Pointers are a special case as arbitrary (valid) pointers can be compared using `std::less<>`.

We provide a single iterator range signature to introduce this facility, but can imagine LEWG wanting to consider sentinels, std ranges support, and bounded output ranges rather than a single iterator to range-check the output.

8 Design choices

8.1 No library support is mandated

It is the intention that this extension be fully backwards compatible, and no library changes are *required*. Library implementers may, if they so desire, take advantage of this feature in order to improve performance, but they are not mandated to do so. This is a conservative position until we have confirmed that there would be no ABI concerns from explicitly applying this to the library specification (see [ABI compatibility](#)).

This extension relies on core language support, but does not change existing program behavior, even if the **trivially relocatable** property is deduced. It merely enables libraries to detect this property, and apply their own optimizations if they so desire.

Looking ahead to a follow-up paper from LEWG that performs a more detailed analysis, a library component (such as a container) will typically be affected in one of three ways:

1. Some components will, based on the definition in the standard, automatically gain **trivial relocatability** when appropriate based on the contained type. Examples are `array`, `pair` and `tuple`.
2. Some components could be marked as unconditionally **trivially relocatable**, if it is desired to do so in the future, for example `shared_ptr` and `filesystem::path`.
3. Some components could be marked as conditionally **trivially relocatable**, based on the **trivial relocatability** of contained types or other conditions. Examples are `optional` and `variant`.

As library implementations vary, the category into which a particular component is placed may vary.

Note that some types that intuitively seem like they might be unconditionally trivially relocatable are often only conditionally trivially relocatable due to forgotten template parameters with default arguments, such as allocators or the deleter policy for `unique_ptr`. In other cases such as `function`, semantic constraints to support small object optimizations can affect the choice. This is part of the reason to focus this proposal exclusively on the minimal core language support, while deferring a detailed library analysis to a follow-up paper for LEWG if this proposal is accepted.

8.2 Contextual keyword vs. attribute

Our design mandates all behavior regarding trivial relocatability rather than leaving potential usage unspecified, as a quality of implementation issue. In particular, several categories of misuse are expected to produce diagnostic errors.

We expect templates to make use of the `trivially_relocatable` markup, and prefer to avoid putting extra work parsing attributes through the template machinery, although there are no technical limitations here. For example, we believe that a specification relying on existing template wording will be simpler than trying to specify a how a pack expansion works within such an attribute (although the groundwork was laid when `alignas` was an attribute).

Usage of the `trivially_relocatable` markup should be clear and simple, especially with its mandated semantics, much as `final` became one of the first contextual keywords. Notably, `trivially_relocatable` would fall into the grammar in exactly the same location as `final` on a class.

By contrast, [P1144R6] prefers to use an attribute. The most obvious benefit is that an unnamed class can unambiguously use the attribute. When using a contextual keyword, we must limit usage to the case disambiguated by the opening paren of the boolean expression.

It has also been pointed out that the use of a parenthetical bool-expression in this position of the contextual keyword grammar might cause problems if some future language extension wanted to place a parenthetical list there, unrelated to contextual keywords:

```
struct Foo { };
struct Foo (Bar) { }; // always a syntax error today, but maybe we'd like to use this tomorrow
struct Foo final { };
struct Foo final (Bar) { }; // always a syntax error today, but maybe we'd like to use this tomorrow
struct Foo trivially_relocatable { };
struct Foo trivially_relocatable (Bar) { }; // uh-oh!
```

Note that this would not be an issue if the hypothetical extension were to place the new parenthetical *before* the contextual keywords, but that is already a constraint on future design. Such concerns do not arise with the attribute form.

We are not aware of any such hypothetical extensions at this point, but should be aware of our choices.

8.3 Type trait vs. concept

Existing library facilities in this space, such as observing trivial copyability, are rendered as type traits rather than concepts. Such type traits can easily be used to constrain templates in `requires` clauses, but do not participate in subsumption relationships.

It would be simple to specify a concept in terms of the proposed trait, but that trait is squatting on the good name. Note that the contextual nature of the keyword means there is no actual conflict here, but overloading an identifier this way might be confusing for users.

The C++ grammar enforces that concepts cannot be specialized, unlike templates. Specifying as a concept, rather than a type trait, would eliminate an unusual source of potential user error, and might have been the preferred approach for this reason, were it not for the precedent of the existing family of trivial type traits.

8.4 `trivially_relocate` as the single place for compiler magic

When it comes to exposing core language facilities as a library API, we prefer to keep the interaction as small and local as possible, ideally just a single “magic” function to imbue the new behavior.

Looking at a more general purpose library interface, we see the importance of being able to relocate arbitrary ranges of objects, using the traditional move-and-destroy semantic where trivial relocatability is not supported. We believe there are sufficient complexity getting the details right when handling overlapping source/destination ranges that it merits adding to the library. Support for overlapping ranges is inspired by `memmove` allowing for overlapping trivially relocatable ranges.

We offer two range APIs. `relocate` accepts pointers to ranges in memory, supports both movable and trivially relocatable types, supports overlapping ranges, and mandates trivial relocation when supported. `move_and_destroy` is directly inspired by `uninitialized_relocate` in [P1144R6], and supports iterator ranges more broadly. As it is not generally possible to identify overlapping ranges where the iterator types vary, we offer no support for overlapping ranges. Unlike [P1144R6], this specification does *not* permit trivial relocation of the elements, guaranteeing the move-and-destroy semantic.

Note that we deliberately use C++20 `requires` clauses to constrain these functions. We believe this is important for the tight core/library specification for `trivially_relocate`, but is largely cosmetic to ease review when

looking at

the other two functions. LWG may prefer to specify such functions with a *Constraints*: function element instead.

9 Known concerns

9.1 Separately managed objects

Performing trivial relocations is generally not appropriate for an object whose lifetime is separately managed, such as a local variable on the stack, an object of static or thread storage duration, or a non-static data member within a class. Adding compiler support to better observe trivial relocations means we may get warnings on such misuse. (This concern is similar to destroying and recreating an object in-place. In such cases it is essential to recreate the object before its destructor will be called implicitly — hence a warning and not an error, as the idiom is already valid.)

9.2 Internal pointers to members

If a user explicitly (and erroneously) marks as **trivially relocatable** a class with an invariant that stores a pointer into an internal structure, then relocation will typically result in UB. For example:

```
class MyClass
trivially_relocatable
{
private:
    int data_v[2];
    int *data_p;    // data_p will not be valid after a trivial relocation.
public:
    MyClass(int a, int b)
    {
        data_v[0] = a;
        data_v[1] = b;
        data_p = &(data_v[1]);
    }
    MyClass(MyClass &&other)
    {
        data_v[0] = other.data_v[0];
        data_v[1] = other.data_v[1];
        data_p = &(data_v[1]);    // NOT copied from other.data_v !!!
    }
};
```

After trivial relocation, `data_p` in the relocated object would point to the address where the member of the old object resided, but that object's lifetime has now ended. UB occurs for any use of that pointer now, other than assigning a new value, or destruction.

Note that this cannot happen without the user explicitly marking the class as **trivially relocatable**, as the default rules for **trivial relocatability** handle this use case by requiring only implicitly defined move constructors.

9.3 Active element of a union

When a union is trivially relocated, the active element of the union must follow along, or it would be undefined behavior to access the relocated active element. As compilers typically do not explicitly track the active member, it is thought that this would have minimal impact on implementations. However, for the purpose of static analysis, or compilers seeking undefined behavior to exploit for optimizations, it is necessary to add the guarantee to propagate the active element through the “compiler magic” in `trivial_relocate` function. Note that this

guarantee must apply to non-static data members that are unions too, including anonymous unions and variant data members.

9.4 ABI compatibility

We do not anticipate any ABI compatibility concerns, but have been surprised before. Once the incubator is happy to forward this proposal to evolution, we will ask the ABI group for their opinion to better inform this part of the paper.

We deliberately avoid applying the `trivially_relocatable` trait to the standard library, deferring that work to a separate paper once the ABI implications are properly understood.

9.5 Relocating const-objects

The specification for a trivially relocatable type supports const-qualified types, including const-qualified class types. However the `trivially_relocate` function itself is constrained to exclude ranges of `const` objects.

The key concern is that destroying non-const objects with automatic, static, or thread storage duration is valid, as long as those objects are replaced before their destruction is invoked. However, it is undefined behavior to replace a `const` object with such a storage duration in the same manner (6.7.2 [intro.object]p10).

In order to protect from accidentally triggering UB, the special function to trivially relocate objects accepts only non-`const` qualified object. If the user knows they are dealing with objects of dynamic storage duration, they can cast away constness before the call with a `const_cast`, but must do so explicitly, acknowledging their intent.

Similarly, `const`-qualified non-static data members satisfy the definition of trivially relocatable, so do not disqualify class types with such non-static data members from also being trivially relocatable, and the complete object can easily (and safely) be relocated without requiring a `const`-cast. This is the same behavior that is supported for references as non-static members.

9.6 Trivially relocatable is not trivially swappable

The most significant difference between this proposal and [P1144R6] is whether the trivial relocatability property is necessary and sufficient to optimize `std::swap` to three relocate operations.

[P1144R6] reports significant benefits optimizing `std::swap` as three relocate operations, which is why it requires trivially relocatable types to have appropriate assignment operators as well. The underlying assumption is that destroy-then-move-construct has the same behavior as move-assign for supported types. This restriction immediately excludes a set of types very important to the authors of this proposal, namely types using scoped allocators (or any other allocator that does not propagate on `swap`). Prominent examples of types that do not satisfy this second move constraint include class types with `const` or reference data members, all polymorphic types, and types with non-propagating allocators (such as all of `pmr`).

The focus of this proposal is purely whether moving a sequence of bytes (once) produces a valid object representation, modelling move-then-destroy for movable types, but also allowing relocation of immovable types with the right properties. Such relocation functionality is essential for maintaining performance in types like `std::pmr::vector` holding allocator aware types that are *not* no-throw movable, such as `std::pmr::vector` itself, and `std::pmr::string`, where the container maintains the invariant that all elements have the same allocator. Note that no-throw movable types are already optimized by the current library `vector`, but `pmr` types are *not* no-throw movable.

One of the underlying assumptions of this paper is that trivial relocation is a fundamental operation in the abstract machine, and trivial swap would be a similar fundamental primitive, much as the library continually runs into the design constraint that `swap` is essentially a primitive operation, which the standard library expresses as a compound operation through multiple moves.

We observe that relocate and swap have very different interfaces, where relocate starts with one valid object, and a region of memory where no object exists. Its end state is the same, but which region of memory holds an actual object has changed. Swap starts and ends with two live objects, and it is only their values that are exchanged.

There are no extensions to the object lifetime model of the abstract machine needed to support (trivial) swap, and indeed, it is valid to implement the “trivial swap” optimization today for trivially copyable types, relying on 6.7.3 [basic.life]p8 notion of transparently replaceable objects, although that would risk undefined behavior when one or both of the objects were subobjects of some other complete objects. We believe [P1144R6] has this same problem swapping subobjects.

It is also worth noting that all trivially swappable types are trivially relocatable, as we can effectively perform a trivial swap, and then end the lifetime of the original object without running its destructor, per the trivial relocate semantics where destructors do not run. The converse is not true though, as we are concerned with a variety of types that would satisfy a primitive trivial relocation specification, but not a primitive trivially swappable specification.

While this proposal does not tackle the problem of `std::swap`, we suggest that it would be better solved by a separate `is_trivially_swappable` trait, that would be defined as the conjunction of `is_trivially_relocatable` and another trait indicating that move-assignment is equivalent to destroy-then-move-construct. We do not have a catchy name for that “sensible” type trait yet though. We believe that the movability trait would be best implemented with another feature like the `trivially_relocatable` specification in order to better deduce the property for common types, according to their base classes and non-static data members. Note that the `is_trivially_swappable` trait would be implemented entirely in terms of the other two traits, and *not* require a third markup of its own. This approach would properly deconstruct the concerns into their separate dimensions. As a further extension, a `trivially_swappable` specification could imply both `trivially_relocatable` and predictable move semantics.

10 Alternative designs

10.1 Smarter default for dependent templates

An alternative design we considered for the `trivially_relocatable` specifier lacking a predicate is that, rather than defaulting to `true`, the predicate would default to `(std::is_trivially_relocatable_v<PACK> && ...)` where `PACK` would be a template parameter pack comprising the (potentially empty) set of types of any dependent bases and non-static data members. Hence, `trivially_relocatable` would be a “make me trivially relocatable if possible” request for class templates, rather than forcing an error on instantiation. It would still be an error to mark a class template having non-depended bases or non-static data members that were, in turn, not trivially relocatable.

We rejected this design as likely to be confusing, ascribing multiple possible meanings to the simple `trivially_relocatable` specifier.

10.2 Ignoring `trivially_relocatable` like `constexpr`

To simplify working with class templates, we considered treating a `trivially_relocatable` specifier that evaluates to `true`, including the default case where the predicate is implicitly `true`, like `constexpr` where it is simply ignored at instantiation time if that class template cannot be made trivially relocatable. This would still be expected to diagnose non-dependent reasons for failure eagerly though, like `static_assert`.

We rejected this direction for additional complexity, and breaking the principle of least astonishment where the value of a `trivially_relocatable` specifier can be relied on as accurate.

11 FAQ

11.1 Is `void` trivially relocatable?

No, and it is not trivially copyable either.

11.2 Are reference types trivially relocatable?

No, and they are not trivially copyable either.

11.3 Why not?!

It is not possible to take the address of a reference to pass it to `relocate`. How the compiler implements references is entirely unspecified, and may not need physical storage if the reference never leaves a local scope. As it is not meaningful to ask about copying/relocating a naked reference, rather than the entity it refers to, these trivial properties are `false`.

11.4 Why can a class with a reference member be trivially relocatable?

For the same reason such a class can be trivially copyable. Strictly speaking, reference members are not non-static data members, and you cannot create a pointer-to-data-member to one. They deliberately fall through the relevant wording by not appearing in the list of disallowed entities, despite not being trivially copyable/relocatable as a distinct type in their own right. This is subtle wording for the unwary, but has been standard practice for many a year.

11.5 Are *cv*-qualified types, notably `const` types, trivially relocatable?

Yes, if the unqualified type is trivially relocatable.

11.6 Can `const`-qualified types be passed to `trivially_relocate`?

No, see [Relocating `const`-objects](#). While `const`-qualified types are trivially relocatable, and so do not inhibit the trivial relocatability of a wrapping type, they are typically not safe to relocate due to leaving behind a dead object that cannot be replaced using well-defined behavior. Hence, the `trivially_relocate` function is constrained to exclude `const`-qualified types. This can be worked around using `const_cast` if doing so would not introduce undefined behavior

11.7 Can non-implicit-lifetime types be trivially relocatable?

Yes. See [New semantics](#)

11.8 Why are virtual base classes not trivially relocatable?

As they are not trivially copyable either. We believe it is possible to implement virtual bases such that trivial copy and relocatability would not be a concern, as all the runtime fix-ups can be resolved in the initial object construction. However, it is not clear that all implementations use such a layout, and forcing trivial operations may be an ABI break.

We would love to remove this restriction, but this should be kept consistent with the corresponding restriction on trivially copyable. If no current ABIs are affected we might consider normatively allowing, or even encouraging, such an implementation (for both trivialities) as conditionally supported behavior on platforms that would not incur an ABI break.

11.9 Why do deleted special members inhibit implicit trivial relocatability?

Initially we considered allowing trivial relocation of types with these special members functions deleted, based on the notion that we are familiar with the idea since C++17, where “mandatory copy elision” started propagating non-copy/movable return values. However, relocation is not the same as the initial construction occurring at a different location (copy elision), so there were objections to the idea that when a user deliberately removes an operation, we should not *silently* re-enable it by a back door. Note that this changes only the default, preventing accidental relocation of non-copyable non-movable types for which relocatability was neither considered nor

intended — if trivial relocatability is desired, such classes can be made **explicitly_trivially_relocatable** by means of the **trivially_relocatable** keyword.

This design also follows that of the core language for trivial copyability, which was changed to exclude types that deleted all copying operations in C++17 ([CWG1734]).

12 Proposed wording

All changes are relative to [N4928].

Editors' note: Uncompleted wording tasks:

- complete specification for **trivially_relocate** (currently a half-specified mess)
- specify support for overlapping ranges in the “equivalent to” wording of the non-trivial **relocate** function

12.1 Feature macros

In 15.11 [cpp.predefined] add the following macro:

```
__cpp_trivial_relocatability          TBD
```

Amend 17.3.2 [version.syn] with the following macro to the header <version>.

```
#define __cpp_lib_trivially_relocatable    TBD    // also in <memory>, <type_traits>
```

12.2 Specification of trivial relocatability

Append to 6.8.1 [basic.types.general]p9:

Scalar types, trivially relocatable class types, and arrays of such types, are collectively called **trivially relocatable types**.

Add a new paragraph to 11.2 [class.prop]:

A **trivially relocatable class** is a **class** that:

- has no base classes that are not of trivially relocatable type,
- has no non-static non-reference data members whose type is not a trivially relocatable type,
- has no virtual base classes,
- has no user-provided or deleted destructors,
- either has no **trivially_relocatable** predicate, or has a **trivially_relocatable** predicate that evaluates to **true**,
- and either
 - has a move constructor that is neither user-provided nor deleted, or
 - has no move constructor and has a copy constructor that is neither user-provided nor deleted.

[Note: accessibility of the special member functions is not relevant — end note]

[Note: trivially copyable class types are implicitly trivially relocatable unless they have a **trivially_relocatable** predicate that evaluates to **false** — end note]

[Note: a type with const-qualified or reference members can be trivially relocatable — end note]

[Note: lambdas are trivially relocatable if and only if their closure type is a trivially relocatable class type — end note]

Design note: It is possible, by means of the **trivially_relocatable(true)** specification, to declare a class as trivially relocatable even if that class has user-provided special members (see proposal).

12.3 Grammar for trivially_relocatable

Add `trivially_relocatable` to the list of *Table 5: Identifiers with special meaning* ([`tab:lex.name.special`] in 5.10 [`lex.name`])

Change the grammar in 11.1 [`class.pre`] to add `class-context-seq`, `class-context-keyword`, `class-triv-reloc-spec` and `class-triv-reloc-expr` as follows:

```
class-head:
- class-key attribute-specifier-seqopt class-head-name class-virt-specifieropt base-clauseopt
- class-key attribute-specifier-seqopt base-clauseopt
+ class-key attribute-specifier-seqopt class-head-name class-context-seqopt base-clauseopt
+ class-key attribute-specifier-seqopt class-triv-reloc-expropt base-clauseopt

+ class-context-seq:
+   class-context-keyword class-context-seqopt

+ class-context-keyword:
+   class-virt-specifier
+   class-triv-reloc-spec

+ class-triv-reloc-spec:
+   trivially_relocatable
+   class-triv-reloc-expr

+ class-triv-reloc-expr:
+   trivially_relocatable ( constant-expression )
```

Add the following paragraph before 11.1 [`class.pre`]p5:

Each form of `class-context-keyword` shall appear at most once in a complete `class-context-seq`.

In a `class-triv-reloc-expropt`, the `constant-expression`, if supplied, shall be a contextually converted constant expression of type `bool` (7.7 [`expr.const`]). The `class-triv-reloc-specopt trivially_relocatable` without a `constant-expression` is equivalent to the `class-triv-reloc-specopt trivially_relocatable(true)`.

EDITORS' NOTE: We probably need to add something similar to p5, or revise p5 to use the new grammar term `class-context-seq` instead, and extend the example.

Add the following paragraphs to 11.2 [`class.prop`]:

A class type having `class-triv-reloc-specopt trivially_relocatable` or `class-triv-reloc-expropt trivially_relocatable` with value `true` specifies that it shall be considered **trivially relocatable** per the proposed definition in 6.8.1 [`basic.types.general`].

It shall be a reportable error and the program shall be ill-formed if a type `T` is declared with the `trivially_relocatable class-triv-reloc-specopt` or the `trivially_relocatable class-triv-reloc-expropt` with value `true` where one or more of the following is true:

- `T` has a virtual base class,
- `T` has a (non-static non-reference) member that is not trivially relocatable, and
- `T` has a base class that is not trivially relocatable.

12.4 New type trait

Add to the `<type_traits>` header synopsis in 21.3.3 [`meta.type.synop`]:

```
template< class T >
struct is_trivially_relocatable;
```

```
template< class T >
inline constexpr bool is_trivially_relocatable_v = is_trivially_relocatable<T>::value;
```

Add a new entry to table 47 in 21.3.5.4 [meta.unary.prop]

Template	Condition	Preconditions
template<class T> struct is_trivially_relocatable;	T is a trivially relocatable type	remove_all_extents_t<T> shall be a complete type or cv-void

12.5 Relocation functions

12.5.1 trivially_relocate

Add to the <memory> header synopsis in 20.2.2 [memory.syn]p3:

```
// 20.2.6, explicit lifetime management template<class T>
T* start_lifetime_as(void* p) noexcept;
template<class T>
const T* start_lifetime_as(const void* p) noexcept;
template<class T>
volatile T* start_lifetime_as(volatile void* p) noexcept;
template<class T>
const volatile T* start_lifetime_as(const volatile void* p) noexcept;
template<class T>
T* start_lifetime_as_array(void* p, size_t n) noexcept;
template<class T>
const T* start_lifetime_as_array(const void* p, size_t n) noexcept;
template<class T>
volatile T* start_lifetime_as_array(volatile void* p, size_t n) noexcept;
template<class T>
const volatile T* start_lifetime_as_array(const volatile void* p,
                                          size_t n) noexcept;
```

```
template <class T>
requires (is_trivially_relocatable_v<T> && !is_const_v<T>)
T* trivially_relocate(T* begin, T* end, T* new_location) noexcept;

template <class T>
requires ((is_trivially_relocatable_v<T> && !is_const_v<T>) ||
          is_nothrow_move_constructible_v<T>)
T* relocate(T* begin, T* end, T* new_location);

template <class InputIterator, class NoThrowForwardIterator>
requires move_constructible<iter_value_t<NoThrowForwardIterator>>
NoThrowForwardIterator move_and_destroy(InputIterator first, InputIterator last,
                                       NoThrowForwardIterator destination);
```

Append to 20.2.6 [obj.lifetime]:

```
template <class T>
requires (is_trivially_relocatable_v<T> && !is_const_v<T>)
T* trivially_relocate(T* begin, T* end, T* new_location) noexcept;
```

Preconditions: end is reachable from begin.

[`new_location`, `new_location` + `begin` - `end`) denotes a region of allocated storage that is a subset of the region of storage reachable through (6.8.4 [basic.compound]) `new_location` and suitably aligned for the type `T`.

Effects: Implicitly creates objects (6.7.2 [intro.object]) within the denoted region consisting of an object *a* of type `T` whose address is `p`, and objects nested within *a*, as follows: The object representation of *a* is the contents of the storage prior to the call to `trivially_relocate`. The value of each created object *o* of trivially-relocatable type `U` is determined in the same manner as for a call to `bit_cast<U>(E)` (22.15.3 [bit.cast]), where *E* is an lvalue of type `U` denoting *o*, except that the storage is not accessed. The value of any other created object is unspecified.

Returns: A pointer to the *a* defined in the Effects paragraph.

Throws:: Nothing.

Remarks: The active member of any union objects or subobjects in the relocated range [`new_location`, `new_location` + `begin` - `end`) is the active member of the corresponding union objects or subobjects from the original range [`begin`, `end`).

Ends the lifetime of the objects in the range [`begin`, `end`) without running their destructors, as if the storage were reused by another object (6.7.3 [basic.life]).

[Note: A likely implementation will simply call a compiler intrinsic that calls `memmove` and updates its notion of the object lifetime. —end note]

EDITOR’S NOTE: THIS IS A LIGHTLY MASSAGED COPY OF `start_lifetime_as` SPEC AND NEEDS MORE WORK, IN PARTICULAR WRT USING `BIT_CAST` TO MAGICALLY IMBUE LIFE INTO NEW OBJECTS

12.5.2 `relocate`

We are also proposing a new “convenience” function template that uses trivial relocatability where available:

```
template <class T>
    requires ((is_trivially_relocatable_v<T> && !is_const_v<T>) ||
              is_nothrow_move_constructible_v<T>)
T* relocate(T* begin, T* end, T* new_location);
```

Effects: Equivalent to:

```
if constexpr (is_trivially_relocatable_v<T>) {
    return std::trivially_relocate(begin, end, new_location);
}
else if (ranges-do-not-overlap && is_nothrow_move_constructible_v<T>) {
    uninitialized_move(begin, end, new_location);
    destroy(begin, end);
    return new_location;
}
else {
    // relocate-and-destroy each member in the appropriate order
    return new_location;
}
```

Throws: Nothing.

12.5.3 `move_and_destroy`

We are also proposing a new “convenience” function template that never uses trivial relocatability, even where available:

```
template <class InputIterator, class NoThrowForwardIterator>
    requires is_nothrow_move_constructible_v<iter_value_t<NoThrowForwardIterator>>
```



```
NoThrowForwardIterator move_and_destroy(InputIterator first, InputIterator last,
                                       NoThrowForwardIterator destination);
```

Precondition: the ranges do not overlap. [Editors' note: sloppy phrasing.]

Effects: Equivalent to:

```
for (; first != last; ++destination, (void)++first) {
    ::new (voidify(*destination)) iter_value_t<NoThrowForwardIterator>(*first);
    destroy_at(addressof(*first));
}
return destination;
```

Throws: Nothing, unless an exception is thrown by a move constructor.

Remarks: If an exception is thrown, all objects in both the source and destination ranges are destroyed.

Editors' note: Review *voidify* in light of C++23 changes.

Editors' note: Preconditions and throws clause implicit from **Effects:**, but stated for clarity while in Evolutionary groups.

Design note: What do we do about `InputIterators` that return proxies? The `addressof` to destroy will not work. Do we add more constraints? Is there a better pattern?

13 Appendix: Comparison with [P1144R6]

Here we perform a detailed comparison of this proposal with [P1144R6], highlighting the large degree of overlap, and describing the differences.

The two papers agree to a large extent on the design space, and largely quibble on the details, sometimes cosmetically, and in a few places technically.

13.1 A Common Basis

Both papers introduce a *trivially relocatable type* as a new type category, and even agree on spelling. Both papers define that category in a way that can be implicitly deduced for many existing types, notably scalars and arrays, and that can recursively deduce that property in classes comprising only trivially relocatable types, and without user-provided special member functions. Both papers provide an explicit markup, using the `trivially_relocatable` token, for users to mark their own classes with user-provided special member functions as retaining this new property. Both markups allow for a boolean predicate for a class to conditionally opt-in to the new property when it is not inferred. Both papers agree that it is undefined behavior to mark up a class whose move constructor and destructor maintain an invariant that does not support simple bitwise movement, such as an internal pointer.

Both papers agree that `memmove` alone is not sufficient to perform an in-memory relocation, as the C++ abstract machine tracks object lifetimes independently of the object representation in memory. Both papers propose standard library APIs to perform a bitwise relocation in a way that is exposed to the abstract machine, and both papers expect that function to be implemented simply as `memmove` at runtime. Both papers provide at least one library API that will safely relocate a range of objects, yet neither adds overloads in the `std::ranges` namespace that rely on range concepts.

Both papers defer on a detailed analysis of the standard library, leaving adoption as an optimization throughout the library as a QoI detail for library implementers.

13.2 Difference in tone

Note that this point is entirely subjective and the opinion of the authors this paper, and not necessarily the author of [P1144R6].

[P1144R6] has a focus on delivering a high-level library interface to users of the language, and proceeds with the minimal level of detail with regard to core specification to make those library APIs implementable. This is evidenced by language talking about the absence of side effects to infer that certain code transformations are possible, spread across a number of functions, and a library API that works with iterator ranges as the most useful approach that fits within the standard library design.

This paper starts from the principle that a change in the abstract machine is necessary, so starts with a core specification that explicitly handles changes to the rules of object lifetimes, and puts all of that abstract machine magic in exactly one function that is available to library implementers and users of the library alike. The broader library support is then built on top of this function, which may be reflected in the design of the library APIs themselves.

13.3 Interface vs. semantics

In [P1144R6], a type supports relocation based upon the *public interface* of its bases and non-static data members, whereas this proposal is based upon the *semantics* of its bases and non-static data members, following the same principles as trivial copyability. For example, a type with a const-qualified non-static data member may be trivially relocatable under this proposal, but not under [P1144R6] as the `const` data member would use copy rather than move semantics. Note that in some (common) cases, such as scalar types, there is no distinction between move and copy semantics, and both proposals agree on the behavior of such types.

Another way this principle applies is that in [P1144R6] it is an error to explicitly mark a type as trivially relocatable if it does not have a public interface that supports regular move semantics; conversely, while this proposal does not automatically infer trivial relocatability for types with deleted move operations (nor does [P1144R6]), it explicitly allows the user to specify trivial relocatability, overriding the default, as long as all of its bases and members are trivially relocatable in turn.

An alternative way to present this is that [P1144R6] provides a library led destructive-move semantic, that is supported only for movable types. This paper proposes a low level language primitive to relocate objects in memory, regardless of their movability.

13.4 Trivial copyability implies trivial relocatability

Both papers explicitly state that a **trivially copyable** type would be **trivially relocatable** by default. However the inverse is specifically *not* true, i.e., a **trivially relocatable** type is not *necessarily* **trivially copyable**.

Only this paper provides a means to mark a **trivially copyable** type as not **trivially relocatable**. This design decision follows from the principle of least astonishment; if the user requests something, absent a good (astonishing!) reason, they should get what they requested. See also [smarter default](#) for our inspiration for why this is the correct choice, even with the absence of specific motivating use cases.

13.5 Implicit trivial relocatability

The conditions for a type to be trivially relocatable by default are similar, but with several notable differences:

- [P1144R6] requires that the copy constructor not be user provided, where this proposal does not care about the copy constructor unless it also serves as the move constructor.
- [P1144R6] requires that there be no user-provided move or copy assignment operators, whereas this proposal does not care about assignment operators. This turns out to be more significant than the authors of this paper first thought, see [Trivially relocatable is not trivially swappable](#).
- [P1144R6] requires that the relevant move/copy constructor and/or destructor be publicly accessible, whereas this proposal states there should be no accessibility restrictions. This difference arises from [P1144R6] explicitly treating `relocate` as move-then-destroy, while this paper leans recursively into the semantics of bases and members, as for trivially copyable types.
- [P1144R6] supports deleted special member functions, whereas this proposal requires a user explicitly declare their type as trivially relocatable if the relevant members are deleted. This difference comes from

this paper trying to follow the users’ intent declared by their syntax in this case, where [P1144R6] is aims to create a user-friendly facility that does not require excessive markup.

- [P1144R6] specifies that relocation attributes are not relevant if a type would be trivially relocatable by default, i.e., the attribute is only opt-in and never opt-out; this proposal requires the compiler to respect a `trivially_relocatable(false)` specification, i.e., the user specification dominates the default semantics.

13.6 Detecting miscategorization

This paper makes it a diagnosable error to attempt to declare a type as trivially relocatable if any of its bases or non-static data members are not, in turn, trivially relocatable. This approach is intended to avoid silently introducing undefined behavior where the compiler cannot see that all moving parts are trivially relocatable types.

[P1144R6] allows users to mark any class as trivially relocatable, as long as it supports public move semantics, even if comprises bases or data members that are not themselves trivially relocatable. This approach is intended to support integration with third party libraries, where the user believes that it would be safe to move the bytes of all the relevant 3rd party data types.

We believe the difference in approach is that the authors of this proposal are familiar with the Bloomberg ecosystem where a significant portion of the code is home grown, and detecting errors (especially those leading to UB) is important. Conversely, [P1144R6] is targeting the wider C++ ecosystem where users will not want to wait on all of their 3rd party libraries to adopt the feature before they can.

13.7 Moved object lifetime and destruction

[P1144R6] describes its proposed `relocate` functions as “equivalent to a move and a destroy”, with *permission* to elide any side effect of move construction and the destructor. The library wording for “equivalent to” is doing a lot of heavy lifting for unspecified compiler magic, spread across many functions:

- How are objects relocated if not by move-and-destroy? (We will assume bitwise copy from here on)
- How are non-trivially copyable objects brought to life after a bitwise relocation?
- How do we signify to the compiler that the originally moved objects lifetime has ended?

This is leaving a lot to library implementers and users to figure out, and hits problems in the abstract machine similar to the motivations for `std::launder`. Although “the standard is not a tutorial”, it is good to have a common understanding when relying on implicit specification. This proposal is therefore explicit and predictable, not deferring to QoI:

- The standard library `relocate` function is explicitly stated to be a `memmove` call to copy the bytes of the object representation
- The **trivial relocation** is mandated, not optional
- The lifetime of an object ends once it is **trivially relocated** from
- The destructor of such an object shall *not* be called
- The compiler magic to imbue life into the relocated objects is limited to a single function
- The user can explicitly request a **trivial relocation** as the “magic” function is specified and publicly available
- No claim is made as to any equivalence between **trivial relocation** and “move and destroy”. As an example the following non-movable class would be trivially relocatable in this proposal but not [P1144R6], assuming the `unique_ptr` type is trivially relocatable:

```
class X
{
    private:
```

```

    const unique_ptr<SomeObject> d_ptr;
public:
    X() : d_ptr(new SomeObject) {}
};

```

13.8 Support for const data members

This proposal has full support for const-qualified non-static data members, where [P1144R6] has limited support. Consider the following example:

```

#include <type_traits>
#include <string>

struct ConstInt {
    const int i;
};
static_assert(std::is_trivially_relocatable_v<ConstInt>);

struct ConstString {
    const std::string s;
};
static_assert(std::is_move_constructible_v<ConstString>);
static_assert(std::is_trivially_relocatable_v<ConstString>);

struct ConstPmrString {
    const std::pmr::string p;
};
static_assert(std::is_move_constructible_v<ConstString>);
static_assert(std::is_trivially_relocatable_v<ConstString>);

```

Note that this all seems to compile in the [P1144R6]-enabled Clang compiler, as well as under the model proposed by this paper. However, let us now write a test driver:

```

#include <type_traits>
#include <string>
#include <memory_resource>
#include <cassert>

struct PmrString {
    std::pmr::string s;
};

struct ConstPmrString {
    const std::pmr::string p;
};

int main() {
    using Alloc = std::pmr::polymorphic_allocator<>;

    std::pmr::monotonic_buffer_resource buffer;

    // Non-const member moves under move
    PmrString a{std::pmr::string{&buffer}};

    assert(a.s.get_allocator() == Alloc{&buffer});
}

```

```

PmrString b = std::move(a);

assert(a.s.get_allocator() == Alloc{&buffer});
assert(b.s.get_allocator() == Alloc{&buffer});

// const member /copies/ under move
ConstPmrString x{std::pmr::string{&buffer}};

assert(x.p.get_allocator() == Alloc{&buffer});

ConstPmrString y = std::move(x);

assert(x.p.get_allocator() == Alloc{&buffer});
assert(y.p.get_allocator() == Alloc{&buffer}); // fail
}

```

This example demonstrates that the regular move construction behavior of `ConstPmrString` is a non-trivial copy. That is not a problem for trivial relocatability under this proposal, which does not make a strict correspondence between trivial relocatability and move-and-destroy. However, this test driver demonstrates the subtlety that users must be prepared for before using the `[[trivially_relocatable]]` trait in [\[P1144R6\]](#).

13.9 New relocation functions

The proposed interface is very different between this and the [\[P1144R6\]](#) proposals.

- [\[P1144R6\]](#) proposes a “one function fits all” `relocate_at` function which can, internally, make use of trivial relocatability for optimization. It also proposes convenience functions `relocate` and `uninitialized_relocate`.
- This proposal has two different functions, `trivially_relocate`, suitable for only trivially relocatable objects, and a convenience function, `relocate`, suitable for `nothrow` movable objects.

Table 2: Usage examples for the two styles of relocation function.
For ease of formatting the ‘end’ parameter has been omitted from
the function calls.

This proposal	[P1144R6]
<pre> { TriviallyRelocatable *tr = new TriviallyRelocatable; NonTriviallyRelocatable *ntr = new NonTriviallyRelocatable; TriviallyRelocatable *trd = malloc(sizeof TriviallyRelocatable); NonTriviallyRelocatable *ntrd = malloc(sizeof NonTriviallyRelocatable); // The following is valid and WILL ALWAYS // use memcpy: static_assert(std::is_trivially_relocatable_v< TriviallyRelocatable>); std::relocate(*tr, trd); // The following is valid and will // use move+destroy: static_assert(!std::is_trivially_relocatable_v< NotTriviallyRelocatable>); std::relocate(*ntr, ntrd); // The following is not valid: static_assert(std::is_trivially_relocatable_v< NotTriviallyRelocatable>); } </pre>	<pre> { TriviallyRelocatable *tr = new TriviallyRelocatable; NonTriviallyRelocatable *ntr = new NonTriviallyRelocatable; TriviallyRelocatable *trd = malloc(sizeof TriviallyRelocatable); NonTriviallyRelocatable *ntrd = malloc(sizeof NonTriviallyRelocatable); // The following is valid, and MAY OR // MAY NOT use memcpy "under the hood": static_assert(std::is_trivially_relocatable_v< TriviallyRelocatable>); std::relocate_at(*tr, trd); // The following is valid, and will // use move+destroy: static_assert(!std::is_trivially_relocatable_v< NotTriviallyRelocatable>); std::relocate_at(*ntr, ntrd); // The following is not valid: static_assert(std::is_trivially_relocatable_v< NotTriviallyRelocatable>); } </pre>

<pre> { TriviallyRelocatable *tr = new TriviallyRelocatable; NonTriviallyRelocatable *ntr = new NonTriviallyRelocatable; TriviallyRelocatable *trd = malloc(sizeof TriviallyRelocatable); NonTriviallyRelocatable *ntrd = malloc(sizeof NonTriviallyRelocatable); // The following is valid and WILL ALWAYS // use memcpy: static_assert(std::is_trivially_relocatable_v< TriviallyRelocatable>); std::trivially_relocate(*tr, trd); // The following is valid and will // use move+destroy: static_assert(!std::is_trivially_relocatable_v< NotTriviallyRelocatable>); std::trivially_relocate(*ntr, ntrd); // The following is not valid: static_assert(std::is_trivially_relocatable_v< NotTriviallyRelocatable>); } </pre>	<pre> { TriviallyRelocatable *tr = new TriviallyRelocatable; NonTriviallyRelocatable *ntr = new NonTriviallyRelocatable; TriviallyRelocatable *trd = malloc(sizeof TriviallyRelocatable); NonTriviallyRelocatable *ntrd = malloc(sizeof NonTriviallyRelocatable); // The following is valid, and MAY OR // MAY NOT use memcpy "under the hood": static_assert(std::is_trivially_relocatable_v< TriviallyRelocatable>); std::relocate_at(*tr, trd); // The following is valid, and will // use move+destroy: static_assert(!std::is_trivially_relocatable_v< NotTriviallyRelocatable>); std::relocate_at(*ntr, ntrd); // The following is not valid: static_assert(std::is_trivially_relocatable_v< NotTriviallyRelocatable>); } </pre>
---	---

13.10 Cosmetic differences

13.10.1 Attributes or keywords to specify trivial relocatability

Both papers propose a new token, `trivially_relocatable`, to specify that a type is trivially relocatable. However, this proposal prefers a contextual keyword whereas [P1144R6] prefers an attribute syntax. The difference is mostly cosmetic and a matter of taste.

The authors of this proposal find the extra 4 characters enclosing the identifier with brackets ugly and distracting, admittedly very subjective. They prefer the specifier following the class name, rather than following the `class` keyword.

Conversely, the attribute works without modification on unnamed classes, and as part of feature adoption, may be silently ignored on recent compilers if they do not understand it when compiling in an older C++ dialect, whereas the contextual keyword would likely be replaced by a feature macro in code that supports a variety of C++ standards, until some future point where older standards no longer matter to a given user base.

13.10.2 Naming

Naming is always cosmetic. Both proposals lean into the term “trivially relocatable” in the same way, in the core language specification and the corresponding type trait. The choice of names for functions varies considerably more.

13.10.3 Focus on different examples

As this paper is focused on a tight core language specification and detecting errors, the examples focus on technical issues on how the feature is used correctly, and errors the compiler is expected to detect. Notably, template interactions are shown in some detail.

[P1144R6] has less to say (by design) about well-formed programs and misuse, and its examples look toward user experience and use cases.

14 Acknowledgements

This document is written in markdown, and depends on the extensions in [Pandoc](#) and [mpark/wg21](#).

The authors would also like to thank Joshua Berne for his assistance in proof reading this paper, especially the proposed Core wording. Also, this paper is greatly improved by feedback from Arthur O'Dwyer, author of [P1144R6], who corrected many bad assumptions we made about his paper, and helped bring the technical differences into focus. We also benefited from several examples he shared to help illustrate those differences and misunderstandings.

15 References

- [CWG1734] James Widman. 2013-08-09. Nontrivial deleted copy functions.
<https://wg21.link/cwg1734>
- [N4928] Thomas Köppe. 2022-12-18. Working Draft, Standard for Programming Language C++.
<https://wg21.link/n4928>
- [P1029R3] Niall Douglas. 2020-01-12. move = bitcopies.
<https://wg21.link/p1029r3>
- [P1144R6] Arthur O'Dwyer. 2022-06-10. Object relocation in terms of move plus destroy.
<https://wg21.link/p1144r6>
- [P2685R0] Alisdair Meredith, Joshua Berne. 2022-10-15. Language Support For Scoped Allocators.
<https://wg21.link/p2685r0>